# Assignment 2, Part 3(b): RTT–Loss AIMD

Ayusha Patra  Kabileswaran Krishna Kumar  Vihaan Kodeboyina  Millan Kumar

## Idea and evidence

Our algorithm, RTT–Loss AIMD (RLA), grows the congestion window when the connection is using it well, backs off when RTT rises, and cuts back more strongly when TCP detects loss. The goal is to get good throughput without building an unnecessarily large queue.

Our measurements suggest that one signal alone is not enough. At the LeaseWeb server in Washington, Reno averaged about 550 Mbit/s with a median RTT of 34 ms, while BBR reached 567 Mbit/s at 31 ms. At the moji server in Paris, CUBIC averaged 214 Mbit/s and Reno only 1.5 Mbit/s, even though their median RTTs were both around 99–101 ms. The Paris BBR test also had a burst of 1,741 retransmissions between samples. These examples motivate using RTT, throughput, and loss together.

## Compared with Reno, CUBIC, and BBR

**Reno** grows its window roughly one segment per RTT in congestion avoidance and halves it on loss. It is simple, but recovery can be slow on high-capacity, long-RTT paths. RLA keeps the simple additive/loss-backoff idea and adds delay and throughput checks; those checks can also make it grow slower.

**CUBIC** uses a time-based cubic growth curve to probe capacity faster on large-window paths. Its loss-based probing can build queues. RLA tries to back off before loss when delay rises, trading some throughput for less queueing.

**BBR** estimates bandwidth and minimum RTT to control its sending rate. It can achieve high throughput without waiting for loss, but depends on the quality of those estimates and can have sharing issues with competing flows. RLA is simpler and uses throughput only to decide whether to grow; it may leave more bandwidth unused.

## How the window changes

**Starting up.** Keep Linux's initial window, normally ten segments, and use standard slow start: increase the window as packets are acknowledged. This avoids spending the transfer at a tiny window.

**Regular updates.** Use a control interval of at least 100 ms and at least the RTT measured at the interval's start. Remember the lowest positive smoothed RTT seen so far as the baseline. Measure goodput as eight times newly acknowledged bytes divided by the actual elapsed time. Compare it with the starting window's approximate supported rate:

$$\text{window rate} = \frac{8 \times \text{cwnd} \times \text{MSS}}{\text{RTT}} \quad \text{bits/s, with RTT in seconds.}$$

Here, cwnd is in segments and MSS is bytes per segment. This rate is only an approximation, not a measurement of the link's capacity.

**Backing off.** If RTT rises more than 25% above the baseline, reduce cwnd by 10% and leave slow start. On a native TCP loss/congestion event, request a threshold of half the window and let Linux handle recovery. Respond once through that recovery mechanism, rather than halving for every retransmitted packet in a burst.

**Growing carefully.** After slow start, add one segment per completed interval only if RTT is less than 10% above baseline, goodput is at least 80% of the window rate, and the sender is constrained by its congestion window. Otherwise hold it steady. The gap between the RTT thresholds avoids switching between growth and backoff for every small RTT change. The throughput check avoids enlarging a window that is not being fully used.

## Terms

| Term | Meaning |
|---|---|
| ACK | An acknowledgment telling the sender that data arrived. |
| cwnd | Congestion window: the limit on data sent but not yet acknowledged, measured here in segments. |
| MSS | Maximum segment size: the number of data bytes in a full TCP segment. |
| RTT | Round-trip time: how long data and its acknowledgment take to travel. We use a smoothed estimate. |
| Baseline RTT | The lowest positive smoothed RTT seen so far; a reference for spotting extra delay. |
| Goodput | The rate of useful data acknowledged, excluding duplicate retransmissions. |
| Queueing | Packets waiting in network buffers, which adds delay and can raise RTT. |
| Slow start | The initial phase where ACKs let the window grow quickly. |
| AIMD | Additive increase, multiplicative decrease: grow by a fixed amount and back off by a fraction. |

## Pseudocode

```
Start with Linux's initial cwnd and slow-start threshold.
Remember the lowest positive smoothed RTT as base_RTT.
At each interval start, save time, ACKed bytes, cwnd, MSS, RTT.
Interval length = max(100 ms, starting RTT).

On a TCP loss/congestion event:
    threshold = max(2, floor(cwnd / 2))
    let Linux handle recovery; discard the current interval

On an ACK outside recovery:
    update base_RTT
    if interval finished and RTT > 1.25 * base_RTT:
        cwnd = max(2, floor(0.90 * cwnd))
        end slow start
    else if in slow start and constrained by cwnd:
        cwnd += newly ACKed segments, up to the threshold
    else if interval finished and RTT < 1.10 * base_RTT:
        if goodput >= 0.80 * window_rate and constrained by cwnd:
            cwnd += 1
    otherwise hold cwnd

    respect Linux's maximum window
    if interval finished, begin a new interval
```

Skip invalid measurements and restart the interval after recovery or an idle restart. Two segments is the normal minimum, but Linux can temporarily use one after a timeout. The thresholds are were chosen to avoid oscillation but also slightly arbitrarily.

## Fairness, stability, and limitations

**Fairness.** With similar RTTs and packet sizes, flows that experience congestion together should move toward equal windows: they add the same amount, while a shared percentage decrease shrinks their window difference. In practice, longer-RTT flows update less often, low-goodput flows may fail the growth check, and RLA may yield to CUBIC or Reno by backing off earlier. Fairness is not guaranteed.

**Stability.** Growth at low delay and backoff at high delay should keep the rate near an operating range on a stable path. Holding between the two thresholds should reduce small fluctuations. We expect some oscillation, rather than exact convergence, especially with delayed feedback or changing traffic.

**Limitations.** We assume usable measurements and a reasonably stable path. Route changes can make the minimum RTT outdated, wireless loss can cause unnecessary backoff, and ACK bursts can distort throughput. The growth check can leave spare bandwidth unused. Our measurements have not tested competing RLA flows or established fairness and stability.

Implementation: RLA kernel module in the project repository.

*AI acknowledgment:* Various LLMs (primarly GPT-6.1 and qwen3.8) helped critique and refine the algorithm design, confirm the analysis of the measurements, and convert our notes into a proper LaTex document. For Assignment 3, they also helped extensively with linking our algorithm to the Linux kernel.